# Guida Utente e di Installazione Semplice — Albero di Regressione

**Progetto:** Albero di Regressione (Sistemi di Regressione Basati su Alberi di Decisione in Java)
**Versione:** 1.0
**Autore:** Marco Diana
**Target:** Utenti finali, studenti, sviluppatori e valutatori di laboratorio

---

## 1. Requisiti di Sistema

Prima di procedere all'installazione e all'avvio del software, assicurarsi che l'ambiente di esecuzione disponga dei seguenti prerequisiti:

| Componente | Versione Richiesta | Note e Verifica |
|---|---|---|
| **Java Development Kit (JDK)** | **Java 21 LTS** (o Java 21+ LTS) | Verificare con `java -version` e `javac -version`. Consigliata la distribuzione Eclipse Temurin. |
| **Apache Maven** | **3.9.x** o superiore | Verificare con `mvn -version`. Necessario per la compilazione e la risoluzione automatica delle dipendenze. |
| **MySQL Server** | **8.0.x** o **8.4.x** | In ascolto sulla porta predefinita `3306`. |
| **IntelliJ IDEA** *(Opzionale)* | Ultimate o Community (2024.x+) | Consigliato per l'importazione automatica del progetto Maven e l'esecuzione tramite IDE. |
| **Sistema Operativo** | Windows 10/11, Linux, macOS | Supporto multipiattaforma garantito da Java. |

---

## 2. Risoluzione Automatica delle Dipendenze con Maven

> 💡 **NOTA FONDAMENTALE PER L'UTENTE: NESSUNA INSTALLAZIONE MANUALE DI LIBRERIE**
>
> **Non è richiesta alcuna installazione manuale o copia di file `.jar` esterni.**
> Il progetto è interamente gestito tramite **Apache Maven**. Quando si compila o si avvia l'applicazione, Maven contatta automaticamente i repository online ufficiali (**Maven Central**) e scarica in background nella cartella cache locale dell'utente ( `~/.m2/repository` su Linux/macOS o `C:\Users\<NomeUtente>\.m2\repository` su Windows) tutti i componenti e le librerie necessarie: 1. **Modulo Database:** il connettore JDBC ufficiale MySQL ( `com.mysql:mysql-connector-j:8.4.0` ) viene reperito e configurato automaticamente nel classpath del modulo `Server` . 2. **Plugin di Documentazione:** il plugin `maven-javadoc-plugin:3.11.2` scarica autonomamente tutti i moduli necessari per generare la documentazione HTML aggregata. 3. **Estensioni Future (Spring Boot, OpenAPI Swagger, Vaadin Flow):**
> Anche per le estensioni Web e GUI, l'utente **non deve installare nulla esternamente**: - Lo stack **Spring Boot** (Starter Web, Data JPA, Validation) viene scaricato interamente da Maven; - I componenti **Swagger UI / SpringDoc OpenAPI** per la documentazione e il collaudo interattivo delle API REST vengono inclusi al volo; - Il framework **Vaadin Flow** gestisce persino il runtime frontend Node/npm/pnpm in modo trasparente e isolato all'interno della cartella di build del progetto, senza richiedere l'installazione preventiva di Node.js sul sistema operativo dell'utente.

Per forzare la risoluzione e il download preliminare di tutte le dipendenze online è sufficiente eseguire dalla radice del progetto:

```
mvn clean dependency:resolve
```

---

## 3. Configurazione Rapida del Database MySQL ( `MapDB` )

Il sistema si connette a un database relazionale denominato `MapDB` tramite l'utente dedicato `MapUser` (con password `map` ).

### 3.1 Inizializzazione Rapida Tramite Script Batch e Cartella `sql/`

Il progetto include nella cartella radice `sql/` gli script SQL ufficiali pronti all'uso: - `sql/01-init-schema.sql` : crea il database `MapDB` e l'utente `MapUser` con password `map` . - `sql/02-sample-data.sql` : crea e popola le tabelle di test `provaC` (15 tuple) e `servo` (167 tuple).

Per eseguire l'inizializzazione automatica del database su MySQL locale: - **Database pulito (senza tabelle demo):** `cmd  init-db.bat` - **Database con tabelle dimostrative ( `provaC` e `servo` ):** `cmd  init-db.bat -prova` *(Su Linux/macOS utilizzare `./init-db.sh` o `./init-db.sh -prova` )*

---

### 3.2 Script SQL Completo (Esecuzione Manuale Alternativa)

Accedere a MySQL da terminale ( `mysql -u root -p` ), da MySQL Workbench o dal tool *Database* integrato in IntelliJ IDEA ed eseguire il seguente script SQL:

```sql
-- 1. Creazione del database
CREATE DATABASE IF NOT EXISTS MapDB
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

-- 2. Creazione dell'utente applicativo e concessione dei privilegi
CREATE USER IF NOT EXISTS 'MapUser'@'localhost' IDENTIFIED BY 'map';
CREATE USER IF NOT EXISTS 'MapUser'@'%' IDENTIFIED BY 'map';

GRANT ALL PRIVILEGES ON MapDB.* TO 'MapUser'@'localhost';
GRANT ALL PRIVILEGES ON MapDB.* TO 'MapUser'@'%';
FLUSH PRIVILEGES;

USE MapDB;

-- 3. Tabella di test 'provaC' (Attributi discreti e continui, 15 esempi)
DROP TABLE IF EXISTS provaC;
CREATE TABLE provaC (
    X VARCHAR(10) NOT NULL,
    Y DOUBLE NOT NULL,
    C DOUBLE NOT NULL
);

INSERT INTO provaC (X, Y, C) VALUES
('A', 2.0, 1.0),
('A', 2.0, 1.0),
('A', 1.0, 1.0),
('A', 2.0, 1.0),
('A', 5.0, 1.5),
('A', 5.0, 1.5),
('A', 6.0, 1.5),
('B', 6.0, 10.0),
('A', 6.0, 1.5),
('A', 6.0, 1.5),
('B', 10.0, 10.0),
('B', 5.0, 10.0),
('B', 12.0, 10.0),
('B', 14.0, 10.0),
('A', 1.0, 1.0);

-- 4. Tabella benchmark 'servo' (Dataset per servomeccanismi industriali)
DROP TABLE IF EXISTS servo;
CREATE TABLE servo (
    motor VARCHAR(10) NOT NULL,
    screw VARCHAR(10) NOT NULL,
    pgain VARCHAR(10) NOT NULL,
    vgain VARCHAR(10) NOT NULL,
    class DOUBLE NOT NULL
);

INSERT INTO servo (motor, screw, pgain, vgain, class) VALUES
('E', 'E', '5', '4', 0.28125095),
```

```
('B', 'D', '6', '5', 0.5062525),
('D', 'D', '4', '3', 0.35625148),
('B', 'A', '3', '2', 5.500033),
('D', 'B', '6', '5', 0.35625148),
('E', 'C', '4', '3', 0.8062546),
('C', 'A', '3', '2', 5.100014),
('A', 'A', '3', '2', 5.7000422),
('C', 'A', '6', '5', 0.76875436),
('D', 'A', '4', '1', 1.0312537),
('B', 'E', '6', '5', 0.46875226),
('E', 'C', '5', '4', 0.39375174),
('B', 'C', '4', '1', 0.28125095),
('E', 'C', '3', '1', 1.1),
('C', 'C', '5', '4', 0.5062525),
('E', 'B', '3', '2', 1.8999897),
('D', 'C', '3', '1', 0.9000011),
('B', 'C', '5', '4', 0.46875226),
('B', 'B', '5', '4', 0.5437528),
('C', 'E', '4', '2', 0.20625044),
('E', 'D', '4', '3', 0.9187554),
('A', 'D', '4', '3', 1.1062483),
('B', 'C', '6', '5', 0.46875226),
('A', 'C', '4', '2', 0.58125305),
('A', 'B', '6', '5', 0.58125305),
('E', 'C', '6', '5', 0.39375174),
('A', 'A', '3', '1', 5.3000236),
('A', 'E', '4', '2', 0.46875226),
('C', 'D', '3', '2', 1.8999897),
('B', 'B', '3', '2', 4.299977),
('B', 'E', '4', '2', 0.35625148),
('B', 'C', '3', '1', 3.899964),
('C', 'E', '4', '1', 0.5437528);
```

---

## 4. Compilazione e Avvio Passo-Passo

### 4.1 Metodo Consigliato: Avvio Rapido Tramite Script Batch (.bat / .sh)

Per eseguire l'intero stack in un singolo comando con verifica prerequisiti, compilazione e apertura automatica del browser:

```
:: Avvio All-in-One con popolamento tabelle demo:
run-local.bat -prova
```

Lo script `run-local.bat`: 1. Inizializza il database `MapDB` su MySQL con le tabelle `provaC` e `servo`. 2. Compila i fat-JAR di `Server` e `Client` se non già presenti. 3. Avvia il server REST su porta `8080` in una finestra console separata. 4. Esegue il

polling finché il server non è operativo. 5. Avvia il client Vaadin Flow su porta `8081` in una finestra console separata e **apre automaticamente il browser** su `http://localhost:8081`.

Sono inoltre disponibili gli script modulari singoli: - `build.bat`: compila con `mvn clean package -DskipTests`. - `start-server.bat`: avvia il server REST su porta `8080`. - `start-client.bat`: avvia il client Web su porta `8081`. - `test.bat`: esegue la suite dei 64 test con Maven.

---

## 4.2 Metodo A: Compilazione ed Esecuzione Manuale da Riga di Comando (CLI)

### 1. Compilazione del Progetto

Posizionarsi nella cartella radice del progetto:

```
cd <directory-del-progetto>
mvn clean compile
```

Il comando compila tutti i sorgenti Java di `Server` e `Client`.

### 2. Avvio del Server

In un primo terminale (PowerShell o Prompt dei Comandi), avviare il Server da riga di comando tramite Maven:

```
cd Server
mvn compile exec:java -Dexec.mainClass="Main"
```

*(Oppure dalla radice del progetto: `mvn -pl Server compile exec:java -Dexec.mainClass="Main"`)* Il Server visualizzerà:

```
Server is listening on port 8080
```

### 3. Avvio del Client

In un secondo terminale separato, avviare il Client da riga di comando:

```
cd Client
mvn compile exec:java -Dexec.mainClass="Client.Main"
```

*(Oppure dalla radice del progetto: `mvn -pl Client compile exec:java -Dexec.mainClass="Client.Main"`)* Il Client si collegherà immediatamente al Server locale sulla porta `8080` mostrando il prompt interattivo.

---

### 4.3 Metodo B: Compilazione ed Esecuzione da IntelliJ IDEA

Grazie alla configurazione nativa Maven, eseguire l'applicazione da IntelliJ IDEA è immediato:

1. **Apertura del Progetto:**
   Avviare IntelliJ IDEA, selezionare *File* > *Open...* e scegliere la cartella radice del progetto (`Albero di regressione (Java)`). IntelliJ riconoscerà automaticamente il `pom.xml` principale e sincronizzerà i moduli `Server` e `Client`.
2. **Avvio del Server:**
   - Navigare nell'albero di progetto a sinistra in:
     `Server > src > main > java > Main.java`
   - Cliccare con il tasto destro sul file o sul metodo `main` e selezionare **Run 'Main.main()'**.
   - Nella console di output inferiore apparirà: `Server is listening on port 8080`.
3. **Avvio del Client:**
   - Navigare in:
     `Client > src > main > java > Client > Main.java`
   - Cliccare con il tasto destro e selezionare **Run 'Client.Main.main()'**.
   - Interagire direttamente nella console integrata di IntelliJ IDEA.

---

### 4.4 Validazione dell'Installazione tramite Test Automatizzati

Per verificare istantaneamente che l'ambiente sia pronto, che il database MySQL `MapDB` sia raggiungibile e che tutti i moduli siano operativi al 100%, è possibile eseguire l'intera suite di test automatizzati:

```
mvn clean test
```

L'output confermerà l'esecuzione con successo di **64 test** (20 nel modulo Client e 44 nel modulo Server) con **0 fallimenti e 0 errori**:

```
[INFO] Results:
[INFO] Tests run: 64, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
---
[INFO] BUILD SUCCESS
```

---

## 5. Manuale d'Uso Passo-Passo con Esempi Reali

Una volta avviato il Client, viene mostrato il menu iniziale:

```
Learn Regression Tree from data [1]
Load Regression Tree from archive [2]
```

### 5.1 Caso d'Uso 1: Apprendimento da MySQL e Salvataggio su File

1. Inserire `1` e premere Invio.
2. Alla richiesta `File name:` inserire il nome della tabella SQL da cui apprendere l'albero, ad esempio:

   ```
   provaC
   ```

3. Il Client invia la richiesta al Server, che:
   - Si connette a MySQL ed estrae le tuple di `provaC`;
   - Apprende l'albero di regressione calcolando gli split ottimi su $X$ e $Y$;
   - **Salva automaticamente** l'albero serializzato nel file binario `provaC.dmp` nella cartella di esecuzione del server;
   - Risponde con `OK` confermando il successo dell'operazione.

Messaggi a schermo:

```
Starting data acquisition phase!
Starting learning phase!
Starting prediction phase!
```

---

### 5.2 Caso d'Uso 2: Caricamento Rapido da Archivio Binario (`.dmp`)

Se l'albero è già stato appreso in una sessione precedente, non occorre interrogare nuovamente il database: 1. Avviare il Client e inserire `2`. 2. Alla richiesta `File name:` inserire: `text  provaC` *(oppure* `provaC.dmp`*)*. 3. Il Server deserializza istantaneamente l'albero dal file su disco e risponde con `OK`. 4. Si entra immediatamente nella fase di predizione.

---

### 5.3 Caso d'Uso 3: Esecuzione Guidata di una Predizione

Durante la fase di predizione, il Server guida l'utente attraverso i bivi di decisione dell'albero indotto.

#### Esempio di Navigazione su Tabella `provaC`:

Il primo nodo di split generato dall'algoritmo separa gli esempi rispetto all'attributo categorico $X$:

```
Starting prediction phase!
QUERY
[0] X = 'A'
[1] X = 'B'
```

1. Se l'utente digita `1` (ramo $X =' B'$), tutti gli esempi del dataset hanno target uguale a `10.0`. L'albero raggiunge immediatamente la foglia e visualizza:

```
Predicted class: 10.0
Would you repeat ? (y/n)
```

2. Rispondendo `y`, si avvia una nuova predizione:

```
Starting prediction phase!
QUERY
[0] X = 'A'
[1] X = 'B'
```

   Questa volta digitiamo `0` (ramo $X =' A'$).

3. Sul ramo $X =' A'$, l'algoritmo valuta l'attributo continuo $Y$ con soglia ottima a $3.0$:

```
QUERY
[0] Y <= 3.0
[1] Y > 3.0
```

4. Se digitiamo `0` (valore $Y \leq 3.0$), il Server restituisce:

```
Predicted class: 1.0
Would you repeat ? (y/n)
```

5. Se digitiamo `1` (valore $Y > 3.0$), il Server restituisce:

```
Predicted class: 1.5
Would you repeat ? (y/n)
```

6. Digitiamo `n` per terminare la sessione di predizione.

---

## 6. Utilizzo dell'Interfaccia Web Vaadin Flow e Documentazione Swagger UI

Oltre alla tradizionale interfaccia a riga di comando (CLI), il sistema include un'interfaccia grafica Web interattiva realizzata con **Vaadin Flow** e una suite di collaudo e documentazione delle API REST con **Swagger UI**.

### 6.1 Porte di Rete e URL dei Servizi

| Servizio | Modulo | Porta | URL di Accesso nel Browser |
|---|---|---|---|
| **Backend REST API** | `Server` | `8080` | `http://localhost:8080/api/trees` |
| **Documentazione Swagger UI** | `Server` | `8080` | `http://localhost:8080/swagger-ui.html` |
| **Specifiche OpenAPI 3 (JSON)** | `Server` | `8080` | `http://localhost:8080/v3/api-docs` |
| **Frontend Web Vaadin Flow** | `Client` | `8081` | `http://localhost:8081` |

---

### 6.2 Avvio del Backend Spring Boot (Server REST)

Aprire un terminale nella cartella radice del progetto ed eseguire:

```
# Opzione 1: Posizionandosi nella sottocartella Server
cd Server
mvn spring-boot:run

# Opzione 2: Dalla radice del progetto multi-modulo
mvn -pl Server spring-boot:run
```

*(In alternativa, è possibile avviare direttamente la classe `server.ServerApplication` da IntelliJ IDEA).*

*Output atteso:*

```
Tomcat started on port 8080 (http) with context path '/'
Started ServerApplication in ... seconds
```

---

### 6.3 Avvio del Frontend Web Vaadin Flow (Client Web)

In un secondo terminale, eseguire:

```
# Opzione 1: Posizionandosi nella sottocartella Client
cd Client
mvn spring-boot:run

# Opzione 2: Dalla radice del progetto multi-modulo
mvn -pl Client spring-boot:run
```

*(In alternativa, è possibile avviare direttamente la classe `Client.ClientApplication` da IntelliJ IDEA).*

*Output atteso:*

```
Tomcat started on port 8081 (http) with context path '/'
Started ClientApplication in ... seconds
```

Una volta avviato, aprire il browser all'indirizzo: 👉 `http://localhost:8081`

---

### 6.4 Guida Passo-Passo all'Interfaccia Grafica Web Vaadin

1. **Inserimento Tabella/File Archivio:**
   - Nel campo di testo principale, inserire il nome della tabella SQL (es. `provaC`) oppure il nome del file `.dmp` salvato in precedenza.
   - Cliccare sul pulsante **"Apprendi da DB"** (blu) per addestrare un nuovo albero da MySQL, oppure su **"Carica da File (.dmp)"** (verde) per deserializzare istantaneamente un modello salvato su disco.
2. **Visualizzazione Multilivello dell'Albero a Schede (TabSheet):**
   - **Scheda "Diagramma ad Albero" (Icona Cluster):** Visualizza l'albero di decisione come un elegante diagramma vettoriale Mermaid SVG interattivo, con nodi di split rettangolari azzurri, archi orientati con le etichette di partizione (es. `= A`, `<= 2.0`) e nodi foglia terminali ellittici verdi con la classe stimata.
   - **Scheda "Regole IF-THEN" (Icona Elenco):** Mostra tutte le regole decisionali estratte formattate come card individuali con pill/badge colorati per ogni condizione atomica (`X = A`, `Y <= 2.0`) e freccia verso la classe predetta.
   - **Scheda "Testo Raw" (Icona Codice):** Fornisce la trascrizione testuale grezza monospace classica generata dal motore di regressione, ideale per verifiche di basso livello e copy-paste.
3. **Navigazione Guidata con Breadcrumb Interattivo:**
   - Il pannello di destra guida l'utente passo-passo attraverso le diramazioni dell'albero.
   - **Breadcrumb Stepper:** Una barra superiore con pill cliccabili (`Radice › Passo 1 › ...`) consente di tornare all'indietro a qualsiasi decisione precedente con un solo click.
   - **Pulsanti di Bivio:** Ogni opzione di split è presentata con un pulsante a larghezza intera provvisto di icona orientata (`ARROW_RIGHT`), indicante l'attributo e la condizione di filtro.
4. **Card Risultato Foglia ad Alta Visibilità:**
   - Al raggiungimento della foglia, il pannello si trasforma in una card evidenziata verde con icona di successo (`CHECK_CIRCLE`), visualizzando il **valore continuo stimato a grandi cifre (36px)** (es. `1.0000`, `10.0000`).
   - Sono mostrate statistiche riassuntive sul percorso decisionale (profondità raggiunta, numero di passi e nome modello) insieme ai pulsanti **"Nuova Predizione"** e **"Indietro di un passo"**.

---

### 6.5 Esplorazione e Collaudo con Swagger UI

Per esplorare o collaudare direttamente gli endpoint REST: 1. Accedere a `http://localhost:8080/swagger-ui.html`. 2. Nella sezione **Regression Tree API**, saranno visualizzati i tre endpoint documentati: - `POST /api/trees/learn` - `POST /api/trees/load` - `POST /api/trees/predict` 3. Cliccare su un endpoint, premere **"Try it out"**, inserire il payload JSON (es. `{"tableName": "provaC"}`) e premere **"Execute"**: la risposta HTTP 200 con il JSON di ritorno sarà visualizzata immediatamente a video.

---

## 7. Risoluzione dei Problemi Comuni (FAQ & Troubleshooting)

### D1: Ricevo `DatabaseConnectionException: Failed to connect to the database`

- **Causa:** Il servizio MySQL non è in esecuzione, oppure è in ascolto su una porta diversa da `3306`.
- **Risoluzione:**
  - Su Windows, verificare da *Gestione Attività* > *Servizi* che il servizio `MySQL80` (o analogo) sia in stato *In esecuzione*.
  - Da terminale PowerShell amministrativo eseguire: `net start MySQL80`.
  - Verificare che le credenziali configurate in `DbAccess.java` (`MapUser` / `map`) corrispondano a quelle create con lo script SQL del Capitolo 3.

### D2: Ricevo `TrainingDataException: La tabella non esiste` o `non contiene tuple`

- **Causa:** Non è stato eseguito lo script SQL per popolare `provaC` o il nome della tabella è stato digitato in modo errato (attenzione a maiuscole e minuscole su Linux).
- **Risoluzione:** Accedere a MySQL ed eseguire `SHOW TABLES IN MapDB;` e `SELECT COUNT(*) FROM MapDB.provaC;`.

### D3: Ricevo `java.net.ConnectException: Connection refused: connect` all'avvio del Client

- **Causa:** Il Client sta tentando di connettersi sulla porta `8080`, ma il `Server` non è ancora stato avviato.
- **Risoluzione:** Avviare prima il Server (`Server/src/main/java/Main.java`), verificare il messaggio `Server is listening on port 8080`, e solo successivamente avviare il Client.

### D4: Ricevo `UnknownValueException: The answer should be an integer between 0 and ...`

- **Causa:** Durante la predizione, è stato inserito un numero negativo o un indice superiore ai rami mostrati dal bivio `QUERY`.

- **Risoluzione:** Inserire esclusivamente uno dei numeri interi visualizzati tra parentesi quadre nel messaggio di opzioni (es. `0` o `1`).

### D5: Generazione della Documentazione Javadoc Aggregata

Per rigenerare la documentazione tecnica API Javadoc completa di tutti i moduli in formato HTML (inclusi i membri privati e le classi interne), eseguire:

```
mvn javadoc:aggregate
```

I file HTML risultanti saranno disponibili nella cartella `docs/javadoc/` e consultabili aprendo `docs/javadoc/index.html` in qualsiasi browser web.